

Dual-Learning Adaptive Large Neighborhood Search for One-Dimensional Bin Packing: Online Operator Selection and Offline Learned Repair

Mohamed El Amine Kherroubi, Rayan Boukakiou, Idriss Yacine Ziadi, Idris Himeur, Adem Abdelhafidh Diar
Higher National School of Computer Science (ESI)
Algiers, Algeria

Abstract—The one-dimensional bin packing problem (1D-BPP) is a strongly NP-hard combinatorial optimization problem requiring a set of n weighted items to be assigned to the minimum number of capacity- C bins. Metaheuristics scale to large instances but rely on hand-crafted operators governed by policies that are fixed prior to execution and remain insensitive to the evolving state of the search. We propose a dual-learning Adaptive Large Neighborhood Search (ALNS) that embeds machine learning at two structurally distinct decision points within a single destroy–repair loop. At the *online* level, a warm-start LinUCB contextual bandit selects the destroy operator from a five-dimensional encoding of the current search state, replacing the static roulette-wheel mechanism of standard ALNS. At the *offline* level, a gradient-boosted trees (GBT) model—trained by behavioral cloning of Best-Fit Decreasing (BFD)—ranks feasible bins during repair, injecting instance-class knowledge without incurring online learning overhead. An ablation study isolates each component’s contribution; a multi-dataset evaluation covers five standard benchmark families (Scholl, Falkenauer, Wäscher, Hard28, 685 instances total). Under a budget of 5,000 ALNS iterations, all four ablation configurations attain a near-optimal mean gap of 0.05 bins on the Scholl-2 50-item slice; the learning components affect only solve time on this slice. In the comparative study the dual-learning ALNS achieves the lowest mean gap (0.05 bins, versus 0.25 for ant colony optimization at approximately $22\times$ the runtime), strictly dominating every classical baseline on solution quality. Across the five full benchmark families the combined method reaches the LB_1 -optimal solution on 86% of Scholl-2 and 61% of Falkenauer-U instances.

Index Terms—bin packing, adaptive large neighborhood search, contextual bandit, LinUCB, gradient boosting, hyper-heuristics, machine learning, combinatorial optimization

I. INTRODUCTION

The one-dimensional bin packing problem (1D-BPP) is a classical combinatorial optimization problem with broad industrial relevance, from logistics and cutting-stock to scheduling and cloud resource allocation. Given n items, each with a positive integer size, and an unlimited supply of identical bins of fixed capacity C , the task is to assign every item to a bin without exceeding capacity while using as few bins as possible. Despite its simple statement, 1D-BPP is strongly NP-hard [4], so exact solvers do not scale to large instances and practitioners rely on heuristics and metaheuristics.

Two families dominate practical 1D-BPP solving. Constructive heuristics such as First-Fit Decreasing (FFD) and Best-Fit Decreasing (BFD) run in $O(n \log n)$ and offer bounded worst-case guarantees—FFD uses at most $\frac{11}{9} \text{OPT} + \frac{6}{9}$ bins [3]—but they produce a single deterministic packing with no further improvement. Metaheuristics—simulated annealing [6], tabu search [7], grouping genetic algorithms [8], and ant colony optimization [9]—explore the solution space iteratively and reach higher-quality packings on hard instances. Their effectiveness, however, hinges on manually designed neighborhood operators and on selection and acceptance policies whose parameters are fixed in advance and applied identically to every instance and at every stage of the search. Crucially, classical metaheuristics do not exploit the information generated during the search itself: neither the time-varying usefulness of an operator nor the structural regularities of good placements are ever learned.

Machine learning (ML) offers a principled way to make these decisions data-driven. Following the taxonomy of Bengio *et al.* [15], ML can be hybridized with metaheuristics in three broad modes: (i) predicting algorithm configuration or parameters offline, (ii) guiding search decisions online from the current state, and (iii) replacing a hand-crafted operator with a learned model. The first mode adapts poorly within a single run, and the third can be data- and compute-intensive in isolation. We therefore combine the second and third modes at the two decisions that most influence ALNS quality: which part of the incumbent solution to destroy, and how to rebuild it.

Learning-augmented metaheuristics have shown strong results on routing and packing problems. Reinforcement learning has been used to select low-level heuristics in hyper-heuristic frameworks, and neural models have been embedded in large neighborhood search—for example Neural LNS for vehicle routing [16]—to learn destruction or repair policies. Contextual bandits, in particular LinUCB [12], [13], provide a lightweight online mechanism for adaptive operator selection with theoretical regret guarantees, while gradient-boosted trees [14] are strong tabular rankers well suited to scoring candidate placements. These ingredients have rarely been combined within a single bin-packing solver.

This paper presents a dual-learning ALNS for 1D-BPP that injects learning at both decision points of the destroy–repair loop. First, an online warm-start LinUCB contextual bandit selects, at every iteration, one of three destroy operators from a five-dimensional description of the current search state. Second, an offline gradient-boosted-trees model, trained by behavioral cloning of BFD, ranks the feasible bins for each displaced item during repair. The two components are independent and can be toggled separately, which lets us isolate their individual and joint contributions through an ablation study. To our knowledge, this two-point integration—online operator selection together with offline learned repair—has not been reported for bin packing.

The remainder of the paper is organized as follows. Section II formalizes the problem, Section III describes the dual-learning ALNS and its two learning components, Section IV reports the experimental protocol, the ablation study, and the multi-dataset and comparative results, and Section V concludes and outlines perspectives.

II. PROBLEM FORMULATION

Let $\mathcal{I} = \{1, \dots, n\}$ be a set of items with strictly positive integer sizes $s_i \in \mathbb{Z}_{>0}$, and let each bin have integer capacity $C \in \mathbb{Z}_{>0}$ with $s_i \leq C$ for all $i \in \mathcal{I}$. Introduce binary decision variables $x_{ij} = 1$ if item i is placed in bin j , and $y_j = 1$ if bin j is used, where $j \in \{1, \dots, n\}$. The 1D-BPP is then:

$$\min \sum_{j=1}^n y_j \quad (1)$$

$$\text{s.t.} \quad \sum_{j=1}^n x_{ij} = 1, \quad \forall i \in \mathcal{I} \quad (2)$$

$$\sum_{i=1}^n s_i x_{ij} \leq C y_j, \quad \forall j \quad (3)$$

$$x_{ij} \leq y_j, \quad x_{ij}, y_j \in \{0, 1\}. \quad (4)$$

Constraint (2) enforces that each item is assigned to exactly one bin. Constraint (3) simultaneously bounds bin load by C and couples assignment variables to bin-usage indicators. Constraint (4) prevents an item from being placed in a bin that is not open. Objective (1) minimizes the number of bins used.

A standard continuous lower bound is obtained by noting that the total item volume must be distributed over bins of capacity C , requiring at least $\sum_i s_i / C$ bins; integrality gives

$$\text{LB}_1 = \left\lceil \frac{\sum_{i=1}^n s_i}{C} \right\rceil. \quad (5)$$

Sharper lower bounds exist—notably the L_2 bound of Martello and Toth [5], which exploits item-size interactions—but LB_1 suffices as the progress indicator used throughout this work. Strong NP-hardness follows from a polynomial reduction from 3-Partition [4], establishing that no pseudo-polynomial exact algorithm exists unless $\text{P}=\text{NP}$ and justifying a metaheuristic treatment for large instances.

III. PROPOSED DUAL-LEARNING ALNS

Figure 1 gives an overview of the dual-learning ALNS. The loop begins with a BFD warm start, then at each iteration builds a five-dimensional context, selects a destroy operator via the LinUCB bandit, rebuilds the partial solution with the GBT repair model, and applies a simulated-annealing acceptance criterion before updating the bandit and adjusting the temperature.

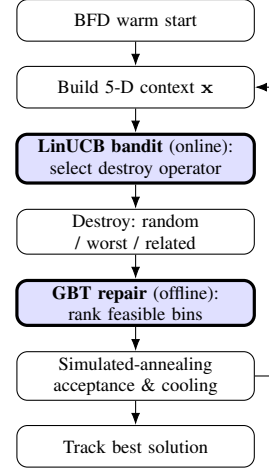


Fig. 1. Architecture of the dual-learning hybrid ALNS. Shaded boxes mark the two machine-learning injection points; the right-hand arrow is the iteration loop.

A. Solution Encoding and Warm Start

A solution is represented as the bijection from items to bins, maintained as two parallel arrays: for each item, the index of its assigned bin, and for each bin, its current aggregate load. Any removal or reinsertion updates both arrays in $O(1)$, and the objective value—the count of non-empty bins—is read directly from the bin-load array without additional traversal.

The search is initialized with Best-Fit Decreasing (BFD). Items are processed in non-increasing size order; each item is placed in the open bin whose residual capacity after placement is minimized, and a new bin is opened only when no feasible bin exists. BFD’s greedy tightness produces an initial packing that is typically within a few bins of the optimum, reducing both the number of iterations required to reach a high-quality region and the frequency of costly total-destruction events.

B. Adaptive Large Neighborhood Search

The search follows the large-neighborhood-search paradigm [10] in its adaptive form [11]. Each iteration (i) *destroys* the incumbent by removing a subset D of items and (ii) *repairs* the partial solution by reinserting D . Because repair can produce any feasible completion, the neighborhood is implicitly exponential in $|D|$, which allows the search to escape local optima unreachable by single-item moves. Three destroy operators form the portfolio: *random* removal of k items; *worst-load* removal, which empties the least-filled bins first; and *related-item* removal, which removes a seed item

together with the items closest to it in size. The destruction radius k is drawn uniformly from $[[0.05n], [0.25n]]$ and is widened toward its upper bound as stagnation grows; all operators guarantee that at least one item remains placed, preventing total destruction.

C. Acceptance Criterion: Simulated Annealing

Candidate solutions are accepted by a simulated-annealing criterion. Let $\Delta = f(x') - f(x)$ denote the change in bin count relative to the incumbent. A candidate with $\Delta \leq 0$ is accepted unconditionally; a candidate with $\Delta > 0$ is accepted with probability $\exp(-\Delta/T)$. The temperature follows a geometric cooling schedule,

$$T \leftarrow \alpha_{\text{cool}} T, \quad T_0 = \frac{1}{\ln 2}, \quad \alpha_{\text{cool}} = 0.9995 \quad (6)$$

so that at T_0 a one-bin worsening is accepted with probability 0.5. Two adaptive mechanisms counteract premature convergence. A *soft reheat* raises T toward $0.35T_0$ when the best solution has not improved for a prescribed number of iterations. A *diversification restart*—reverting to the best solution found, partially reheating to $0.20T_0$, and resetting the stagnation counter—is triggered when the soft reheat fails to produce improvement within a secondary patience budget. The fixed iteration count is the sole termination criterion; no wall-clock time limit is imposed.

D. Online Learning: Warm-Start LinUCB Operator Selection

Operator selection is delegated to a warm-start contextual bandit. During the first 300 selections, a context-free Beta-Bernoulli Thompson sampler quickly identifies promising operators, mitigating the cold-start of a linear bandit on short runs. Thereafter a disjoint LinUCB bandit [13] takes over: for a context vector $\mathbf{x} \in \mathbb{R}^5$ it selects the operator that maximizes

$$\hat{\theta}_k^\top \mathbf{x} + \alpha \sqrt{\mathbf{x}^\top A_k^{-1} \mathbf{x}}, \quad \hat{\theta}_k = A_k^{-1} \mathbf{b}_k, \quad (7)$$

with $\alpha = 0.3$, updating A_k^{-1} by the $O(d^2)$ Sherman–Morrison formula and $\mathbf{b}_k$ by $\mathbf{b}_k \leftarrow \mathbf{b}_k + r \mathbf{x}$. The context encodes the normalized temperature T/T_0 , the stagnation progress toward the next restart, the cost-to-bound ratio $f(x)/\text{LB}_1$, the destruction radius k/n , and the overall search progress. The reward is $\min(1, \text{bins_saved}/\text{gap})$ when bins are saved—normalized by the gap to LB_1 so that improvements near the optimum count more—0.2 for an accepted non-improving move, and 0 for a rejected one.

E. Offline Learning: GBT Repair by Behavioral Cloning

Repair reinserts displaced items in non-increasing size order. For an item, the set of feasible bins (those with enough residual capacity) is scored by a gradient boosted classifier [14], and the item is placed in the highest-scoring bin; a new bin is opened only when no bin is feasible. The model is trained offline by behavioral cloning of BFD: each feasible (item, bin) pair is labeled 1 if BFD would select that bin and 0 otherwise. Every pair is encoded by eleven features normalized by capacity or instance size—item size and its square, size

rank, fraction of items still to reinsert, bin load, residual capacity, post-placement slack, bin occupancy, the largest and smallest item already in the bin, and the fill ratio. Training pools two trace types—fresh BFD replays and post-destruction repair states—to reduce the distribution shift between offline replay and the mid-search states encountered at inference. A standard scaler is stored with the model, and a fast predictor applies it in NumPy and reads class probabilities directly, avoiding per-call validation overhead. Algorithm 2 summarizes the complete loop.

F. Parameter Tuning

Table I lists the nine parameters controlling the solver’s cooling schedule, destruction radius, reheat thresholds, and bandit behaviour. Tuning is handled by a three-stage Optuna [20] pipeline that narrows the search progressively, moving from broad exploration to focused refinement.

TABLE I
TUNABLE HYPERPARAMETERS, THEIR DEFAULTS, AND STAGE 1 SEARCH RANGES. STAGE 2 WINDOWS ARE COMPUTED DYNAMICALLY AS ± 15 –25% OF THE STAGE 1 BEST. LOG-SCALE SAMPLING IS APPLIED WHERE THE EFFECT SPANS ORDERS OF MAGNITUDE.

Parameter	Default	Stage 1 range
α_{cool}	0.9995	(0.998, 0.9999)
bandit_alpha	0.30	(0.05, 1.0) log
initial_temperature	$1/\ln 2$	(0.5, 5.0) log
$k_{\min}$	$0.05n$	($0.01n$, $0.20n$)
$k_{\max}$	$0.25n$	($0.10n$, $0.40n$)
no_improve_frac	0.05	(0.01, 0.20)
reheat_soft_mult	0.35	(0.10, 0.90)
reheat_hard_mult	0.20	(0.05, 0.80)
warmup_calls	300	[50, 600]

The parameters govern five distinct aspects of the solver. α_{cool} and *initial_temperature* control the SA cooling schedule; *reheat_soft_mult* and *reheat_hard_mult* set the temperature multipliers applied at soft and hard reheats respectively. $k_{\min}$ and $k_{\max}$ bound the destruction radius, while *no_improve_frac* defines the stagnation budget that triggers a reheat. Finally, *bandit_alpha* and *warmup_calls* govern the LinUCB exploration bonus and the warm-up length before the bandit takes control.

1) *Composite Objective and Instance Bank*: Each configuration is evaluated on a fixed bank of 100 instances drawn from five benchmark families — Falkenauer-U, Falkenauer-T, Scholl-1, Scholl-2, and Scholl-3 — covering a representative range of problem sizes and structures. The same bank is reused across all evaluations, for consistent and comparable rankings. Configurations are scored by a composite objective that balances solution quality against computational cost:

$$\Phi(\theta) = w_q \cdot \frac{1}{N} \sum_{i=1}^N \frac{b_i(\theta) - \text{LB}_{1,i}}{\max(\text{LB}_{1,i}, 1)} + w_t \cdot \frac{1}{N} \sum_{i=1}^N \frac{t_i(\theta)}{\max(\tau, 10^{-9})}, \quad (8)$$

where $b_i(\theta)$ is the bin count, $\text{LB}_{1,i}$ the continuous lower bound, $t_i(\theta)$ the wall-clock time, and $\tau = 60$ s the time

```

1:  $x \leftarrow \text{BFD-WARMSTART}(); x^* \leftarrow x$ 
2: for  $t = 1$  to  $\text{max\_iter}$  do
3:    $\mathbf{x}_{\text{ctx}} \leftarrow \text{CONTEXT}(x, T, t)$ 
4:    $a \leftarrow \text{BANDIT.SELECT}(\mathbf{x}_{\text{ctx}})$ 
5:    $(\hat{x}, D) \leftarrow \text{DESTROY}_a(x, k)$ 
6:    $x' \leftarrow \text{REPAIRGBT}(\hat{x}, D)$ 
7:   accept  $x'$  by simulated annealing; update  $x^*$ 
8:    $\text{BANDIT.UPDATE}(a, \mathbf{x}_{\text{ctx}}, r)$ ; cool/reheat  $T$ 
9: end for
10: return  $x^*$ 

```

Fig. 2. Dual-learning hybrid ALNS.

normaliser. Weights $w_q = 0.9$ and $w_t = 0.1$ prioritise solution quality while penalising impractically slow configurations. The relative gap formulation makes the quality term scale-invariant across instances of different sizes.

2) *Three-Stage Pipeline*: The pipeline balances broad exploration with precise refinement through a structured three-stage handoff. Stage 0 evaluates parameter groups independently to construct an isolation anchor, which seeds Stage 1’s global Bayesian search without restricting its domain. Stage 1 then hands off to Stage 2 by physically contracting the search boundaries around its best result.

a) *Stage 0: Isolation Grid Search.*: Parameters are split into three functional groups (SA thermal, destruction, and bandit) and grid-searched independently, with all others held at their defaults. The best setting from each group is merged into the *isolation anchor*, passed to Stage 1 as a seed.

b) *Stage 1: Coarse Bayesian Search.*: TPE explores the full search space, seeded by both the solver defaults and the isolation anchor. Structurally invalid pairs (e.g. $k_{\min} \geq k_{\max}$) are pruned before consuming any evaluation budget.

c) *Stage 2: Fine Bayesian Search.*: The search boundaries are contracted to a ± 15 –25% window around the Stage 1 best. TPE refines within this focused basin to produce the final tuned parameter set.

IV. EXPERIMENTS, RESULTS AND ANALYSIS

A. Protocol and Setup

All runs use a fixed random seed (42), a budget of 5000 ALNS iterations, an initial temperature $T_0 = 1/\ln 2$, a cooling rate $\alpha_{\text{cool}} = 0.9995$, and a single pre-trained repair model shared across every dataset. The solver is implemented in Python 3.13 with scikit-learn [19] for the gradient-boosted repair model, and experiments were run on a workstation running Windows 11 with an Intel Core i9-13950HX CPU and 64 GB of RAM. Solution quality is measured by the gap to the lower bound, $\text{gap} = \text{bins used} - \text{LB}_1$ of (5); a gap of zero certifies optimality with respect to LB_1 .

B. Datasets

We evaluate on four widely used 1D-BPP benchmark families, summarized in Table II. Scholl-2 contains uniformly distributed instances with capacity 1000 and 50–500 items. Falkenauer-T comprises triplet instances, where an optimal

bin holds one large and two small items, at capacity 1000. Falkenauer-U holds uniform instances with the tighter capacity 150. Wäscher contributes hard cutting-stock-style instances at capacity 10000, and Hard28 a set of difficult 160–200-item instances at capacity 1000.

TABLE II
BENCHMARK DATASETS USED IN THE EVALUATION.

Dataset	Capacity	Items	Structure
Scholl-2	1000	50–500	uniform
Falkenauer-T	1000	60–501	triplets
Falkenauer-U	150	120–1000	uniform
Wäscher	10000	varied	cutting-stock
Hard28	1000	160–200	hard

C. Parameter Tuning Results

Due to computational constraints, we run a reduced version of the three-stage pipeline: 12 coarse trials (Stage 1) followed by 10 fine-refinement trials (Stage 2), each evaluated on a 10-instance bank subset at 1500 ALNS iterations. The isolation pre-screening and full 100-instance bank are omitted. A larger budget would allow the full pipeline to explore more thoroughly and converge to a significantly better-tuned configuration.

TABLE III
TUNING TRAJECTORY: COMPOSITE OBJECTIVE Φ (8), MEAN RELATIVE GAP TO LB_1 , NORMALISED TIME, AND MEAN WALL-CLOCK TIME AT EACH PIPELINE STAGE.

Stage	Φ	Mean gap	Norm. time	Time (s)
Default parameters	0.0625	0.00590	0.5720	34.32
Stage 1 (coarse)	0.0523	0.00641	0.4655	27.93
Stage 2 (fine)	0.0508	0.00690	0.4459	26.75
Final re-evaluation	0.0528	0.00690	0.4658	27.95

Table III traces the pipeline from the default configuration through to the final re-evaluation. The default row serves as the baseline reference. Stage 1 broadly explores the full S_1 ranges and already reduces Φ by 16%. Stage 2 refines within a contracted window around the Stage 1 best, achieving the lowest observed Φ of 0.0508. The final re-evaluation runs the Stage 2 configuration on a fresh, independent draw from the same bank; the small increase from 0.0508 to 0.0528 reflects the expected stochastic variance of the solver across runs, and remains well within acceptable noise at this evaluation budget. The final tuned configuration is:

$\alpha_{\text{cool}} = 0.9979$, $k_{\min} = 0.009n$, $k_{\max} = 0.097n$,
 $\text{bandit_alpha} = 0.090$, $\text{warmup_calls} = 138$,
 $\text{no_improve_frac} = 0.183$, $\text{reheat_soft_mult} = 0.810$,
 $\text{reheat_hard_mult} = 0.480$, $T_0 = 0.959$.

D. Ablation Study

To isolate the effect of each learning component we run four configurations on the first 20 Scholl-2 50-item instances, keeping all ALNS settings identical and toggling only the two switches (Table IV, Fig. 3). With a 5000-iteration budget all four configurations reach the same near-optimal mean gap of 0.05 bins on this slice, so the contributions of the two

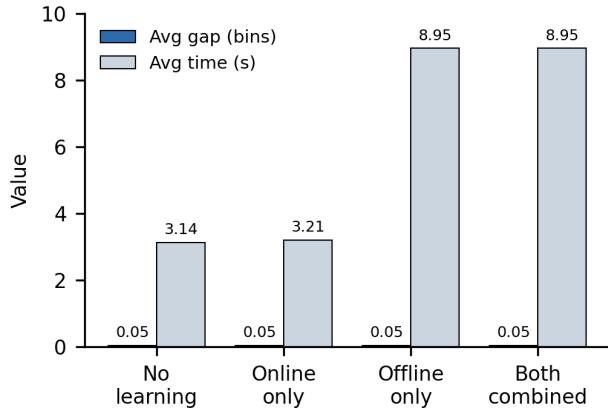


Fig. 3. Ablation on Scholl-2 (5000 iterations, 20 instances): the four configurations reach the same near-optimal mean gap (0.05); the offline repair model adds about 5.8 s of per-placement scoring without quality gain on this slice.

learning components are visible only in solve *time*, not in solution quality. The no-learning baseline—random operator choice and best-fit repair—is the fastest at 3.14 s. Enabling the online bandit alone adds only 0.07 s, while enabling the offline repair model adds 5.82 s, so the per-placement scoring of the repair model dominates the cost. On this small, near-optimal slice the learning components add overhead without quality gain; their benefit is expected on harder instances with larger feasible-bin sets, where the offline model can distinguish placements that best-fit repair cannot (Tables V and VI).

TABLE IV
ABLATION ON SCHOLL-2 (50 ITEMS, 20 INSTANCES).

Configuration	Avg bins	Avg gap	Fill (%)	Time (s)
No learning	17.4	0.05	95.84	3.136
Online only	17.4	0.05	95.84	3.208
Offline only	17.4	0.05	95.84	8.954
Both combined	17.4	0.05	95.84	8.950

E. Multi-Dataset Results

Table V reports the combined method across the five *full* benchmark families (685 instances in total) with per-instance time limits scaled to a 5000-iteration budget. The solver reaches LB_1 on 86% of Scholl-2 instances (mean gap 0.28) and 61% of Falkenauer-U instances (mean gap 0.45). The triplet instances of Falkenauer-T are the hardest relative to LB_1 (mean gap 1.06, no optimum reached), consistent with the known weakness of the continuous bound on triplet structures, where the true optimum typically exceeds LB_1 by one bin. Mean runtime grows steeply with item count—about 33 s on Scholl-2 but more than 1200 s on the 160–200-item Hard28 family—reflecting the cost of model-guided repair over larger feasible-bin sets at 5000 iterations. Instances that exceeded their per-instance time budget are reported with the solver’s best result at the deadline, an honest upper bound rather than the lower bound.

TABLE V
COMBINED METHOD ACROSS THE FIVE FULL DATASETS (685 INSTANCES TOTAL) WITH PER-INSTANCE TIME LIMITS.

Dataset	N	Avg gap	Opt (%)	Avg time (s)
Scholl-2	480	0.28	85.8	33.37
Falkenauer-T	80	1.06	0.0	53.48
Falkenauer-U	80	0.45	61.3	541.49
Wäscher	17	0.82	17.6	28.39
Hard28	28	0.82	17.9	1286.82

F. Comparative Study

Table VI compares the dual-learning ALNS with classical baselines implemented in the same framework, all run in a single session on the hardware above so that runtimes are directly comparable. Exact methods (branch-and-bound, dynamic programming) are omitted because they are intractable at $n = 50$. The pure constructive heuristics FFD and BFD are instantaneous but leave a mean gap of 1.40 bins, and the standalone trajectory metaheuristics—simulated annealing and tabu search—do not improve on construction within their default budgets on this slice. The population methods are stronger: the genetic algorithm reaches a mean gap of 0.90 in 1.11 s and ant colony optimization 0.25 in 193.91 s. The dual-learning ALNS attains the best mean gap of 0.05 in 8.64 s, dominating every baseline on quality while running about $22\times$ faster than ant colony optimization. The metaheuristic baselines are stochastic, so their gaps vary across runs; a single aligned run is reported.

TABLE VI
COMPARISON ON SCHOLL-2 (50 ITEMS, 20 INSTANCES): MEAN GAP TO LB_1 AND MEAN SOLVE TIME.

Method	Avg gap	Time (s)
FFD / BFD	1.40	< 0.01
Simulated annealing	1.40	0.33
Tabu search	1.40	0.22
Genetic algorithm	0.90	1.11
Ant colony	0.25	193.91
Dual-learning ALNS	0.05	8.64

V. CONCLUSION AND PERSPECTIVES

We presented a dual-learning Adaptive Large Neighborhood Search for the one-dimensional bin packing problem that injects machine learning at two decision points of the destroy–repair loop: an online warm-start LinUCB bandit for destroy-operator selection and an offline gradient-boosted repair model trained by behavioral cloning of Best-Fit Decreasing. An ablation isolating the two components shows that with a 5000-iteration budget all four configurations reach the same near-optimal mean gap of 0.05 bins on the Scholl-2 slice, so the contributions of the two learning components are visible there only as solve-time overhead. On the comparative study the dual-learning configuration achieves the best gap (0.05), dominating every baseline on quality while running about $22\times$ faster than ant colony optimization. Across the five full benchmark families (685 instances) the combined method reaches the optimum on 86% of Scholl-2 and 61% of Falkenauer-U

instances; harder families remain bounded by the per-instance time budget. The main limitations are the single-run reporting of stochastic baselines and the runtime overhead of the offline model on the largest instances. Future work will extend the evaluation to full datasets and stronger lower bounds such as the Martello–Toth L_2 bound, where learned repair is expected to pay off on larger feasible-bin sets, replace the cloned repair model with a policy learned directly from search reward, and study the transfer of the learned components across instance families.

REFERENCES

- [1] E. G. Coffman, M. R. Garey, and D. S. Johnson, “Approximation algorithms for bin packing: a survey,” in *Approximation Algorithms for NP-Hard Problems*, 1996, pp. 46–93.
- [2] M. Delorme, M. Iori, and S. Martello, “Bin packing and cutting stock problems: Mathematical models and exact algorithms,” *Eur. J. Oper. Res.*, vol. 255, no. 1, pp. 1–20, 2016.
- [3] D. S. Johnson, “Near-optimal bin packing algorithms,” Ph.D. dissertation, MIT, Cambridge, MA, 1973.
- [4] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: Freeman, 1979.
- [5] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. New York, NY: Wiley, 1990.
- [6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [7] F. Glover, “Future paths for integer programming and links to artificial intelligence,” *Comput. Oper. Res.*, vol. 13, no. 5, pp. 533–549, 1986.
- [8] E. Falkenauer, “A hybrid grouping genetic algorithm for bin packing,” *J. Heuristics*, vol. 2, no. 1, pp. 5–30, 1996.
- [9] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA: MIT Press, 2004.
- [10] P. Shaw, “Using constraint programming and local search methods to solve vehicle routing problems,” in *Proc. Int. Conf. Principles and Practice of Constraint Programming (CP)*, 1998, pp. 417–431.
- [11] S. Ropke and D. Pisinger, “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *Transp. Sci.*, vol. 40, no. 4, pp. 455–472, 2006.
- [12] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 661–670.
- [13] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, “Contextual bandits with linear payoff functions,” in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 208–214.
- [14] J. H. Friedman, “Greedy function approximation: a gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [15] Y. Bengio, A. Lodi, and A. Prouvost, “Machine learning for combinatorial optimization: a methodological tour d’horizon,” *Eur. J. Oper. Res.*, vol. 290, no. 2, pp. 405–421, 2021.
- [16] A. Hottung and K. Tierney, “Neural large neighborhood search for the capacitated vehicle routing problem,” in *Proc. 24th Eur. Conf. Artificial Intelligence (ECAI)*, 2020, pp. 443–450.
- [17] A. Scholl, R. Klein, and C. Jürgens, “BISON: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem,” *Comput. Oper. Res.*, vol. 24, no. 7, pp. 627–645, 1997.
- [18] P. Schwerin and G. Wäscher, “The bin-packing problem: A problem generator and some numerical experiments with FFD packing and MTP,” *Int. Trans. Oper. Res.*, vol. 4, no. 5–6, pp. 377–389, 1997.
- [19] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [20] T. Akiba *et al.*, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631.